

Predicting Hotel Booking Cancellations

Dataset Overview

- **Dataset Name:** Hotel Booking Demand
 - **Source:** [Kaggle Dataset by Jesse Mostipak](https://www.kaggle.com/datasets/jessemostipak/hotel-booking-demand?resource=download)
<https://www.kaggle.com/datasets/jessemostipak/hotel-booking-demand?resource=download>

Introduction

Hotel booking cancellations in the hospitality industry present multiple difficulties, including financial loss, suboptimal use of resources, and low customer satisfaction. They interfere with operational strategies and can result in idle rooms, which are not favourable financially. To avoid these problems, predictive analytics can be adopted to identify causes of cancellations so that hotels can likely take measures that will prevent these events.

This project uses the Hotel Booking Demand dataset obtained from Kaggle that offers important data regarding bookings of hotels, their characteristics, customers' preferences, financial aspects, and cancellation information. This dataset was useful to comprehend the peculiarities of hotel booking and understand the main causes of cancellation.

The main objective of this work is to create a classification model that has the potential to accurately classify a booking as either likely to be cancelled or not, using features including lead time and demographics of the customer, source of the booking, and customer's preferred type of room. By accurately predicting cancellations, hotels can improve their overbooking mechanisms and techniques, improve resource utilization and raise customer satisfaction level by intervention.

Motivated by the exploration of data-driven methodologies for understanding and controlling hotel booking cancellations, the research presented here intends to propose a sound framework for decision-making.

Attributes:

- **Booking Details:** Lead time, arrival date, number of adults/children/babies, and stay duration.
- **Guest Preferences:** Type of room reserved, meal type, and special requests.
- **Booking Source:** Distribution channel, market segment, and agent.

- **Financial Details:** Average daily rate, required deposit, and previous cancellations.
- **Cancellation Status:** Binary variable indicating if a booking was cancelled (target variable).

Dataset Overview

1. Import Libraries, Upload Dataset and Data Loading and Cleaning

```
1. Import Libraries

[9] import pandas as pd
data = pd.read_csv('hotel_bookings.csv')

2. Upload Dataset

[10] from google.colab import files
      uploaded = files.upload()

Choose a file: No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

3. Data Loading and Cleaning

[13] hotel_bookings = pd.read_csv('hotel_bookings.csv')

      print(hotel_bookings.head())

      print("Null values:\n", hotel_bookings.isnull().sum())

      hotel_bookings = hotel_bookings.dropna()

      print("Duplicate rows:", hotel_bookings.duplicated().sum())
      hotel_bookings = hotel_bookings.drop_duplicates()

      print("Dataset shape after cleaning:", hotel_bookings.shape)
```

The code starts with importing a useful library, pandas, which is the most used library for data handling and dealing with data in the form of tables. It then goes on to upload the dataset, this is a dataset format suitable for a Google colab platform to enable the user to upload their own data file. The first cleaning steps are as follows after the dataset is loaded up with libraries and correspondingly imported. The first function of the R program imports the dataset in a more structured form and gives a brief overview of the data by displaying the initial values of the data set. After this, it clears out any rows with missing values thereby capturing the quality of the data set. Then, the following lines of code are used to identify duplicate rows and exclude such rows from subsequent analysis to prevent statements' biases. Last, the shapes of the cleaned dataset are printed, ensure the dataset is prepared on exploratory data analysis and other subsequent modelling phases.

2. One-Hot Encoding and Feature and Target Definition

4. One-Hot Encoding

```
[14] data = pd.get_dummies(data, columns=['hotel', 'meal', 'market_segment'], drop_first=True)
```

5. Feature and Target Definition

```
[15] from sklearn.model_selection import train_test_split

X = data[['lead_time', 'adults', 'children', 'previous_cancellations']]
y = data['is_canceled']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

The code displayed focuses on two key steps in data preprocessing and preparation for machine learning:

One-Hot Encoding

The first section involves using one-hot encoding for categorical inputs in the dataset; these include hotel, meal, and market segment. One-hot encoding transforms these categorical variables into an applicable numerical format where each category is created into columns of binary form. The parameter `drop_first=True` is used to set one missing in a multiple system of variables to avoid multicollinearity. This is a very important phase for the purpose of converting categorical data into a format that ‘models’ can easily understand.

Feature and Target Definition

The second part established the features (X) and the target for the machine learning model (y). These of them are `lead_time`, `adults`, `children`, and `previous_cancellations`, selected as predictive attributes of the consequential cancellations. In this case, the dependent variable Y-sidebar is whether the booking in question was canceled or not (binary variables). The dataset is then divided into the training and testing sets with equal proportions, as the elements defining `test_size` are 0.3. `Random_state = 42` means that the splitting of the data set is repeatable and provides the same values during the test. All these steps are necessary in order to ensure the data optimization in the way it is going to be used for modeling and evaluation.

3.Logistic Regression Model

```
[53]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

logreg = LogisticRegression()
logreg.fit(X_train, y_train)
```

LogisticRegression

The following code sets out a logistic regression as an algorithm to predict hotel bookings cancelation. It divides the dataset for the test set (size 30%) and the training set (size 70%) The logistic regression model is then trained using X_train, y_train.

4.Model Evaluation: Logistic Regression

```
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

# Predictions
y_pred = logreg.predict(X_test)

# Metrics
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print("ROC AUC:", roc_auc_score(y_test, logreg.predict_proba(X_test)[:, 1]))
```

	precision	recall	f1-score	support
0	0.88	1.00	0.93	50
1	0.00	0.00	0.00	7
accuracy			0.88	57
macro avg	0.44	0.50	0.47	57
weighted avg	0.77	0.88	0.82	57

```
[[50  0]
 [ 7  0]]
ROC AUC: 0.47285714285714275
```

This code provides the performance evaluation of the logistic regression model. It generates predictions using the test set (X_test) and computes three metrics: the evaluation metrics of which are the following: the classification report indicating the precision, recall, F1-score; the confusion matrix, which contains information about true positives, true negatives, false positives, and false negatives; the ROC AUC score, which defines the possibility of class distinction. Recall that the positive class (1) is problematic for the model, where recall and F1-score are 0, and ROC AUC is 0.47 which is not acceptable for good classifier.

5.Decision Tree Classifier

```
22] from sklearn.tree import DecisionTreeClassifier

dtree = DecisionTreeClassifier(max_depth=5, random_state=42)
dtree.fit(X_train, y_train)
```

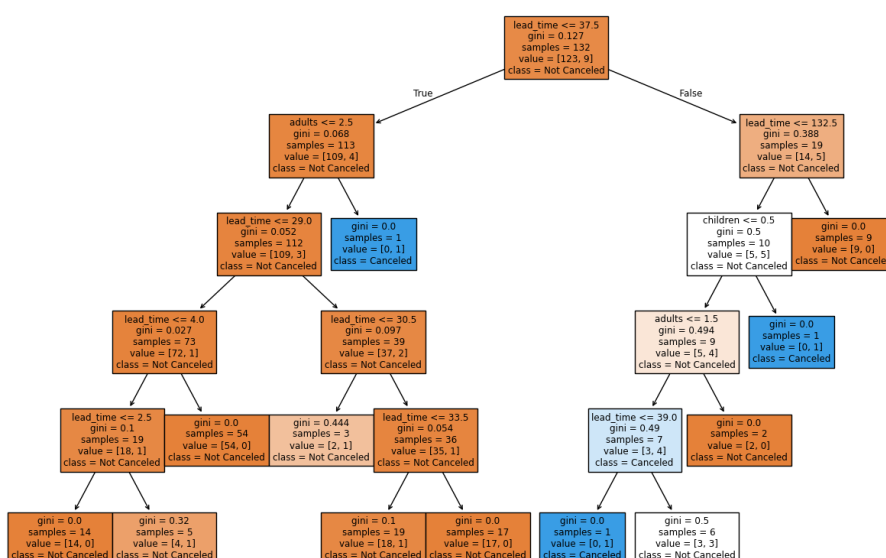
DecisionTreeClassifier

DecisionTreeClassifier(max_depth=5, random_state=42)

In this code is a decision tree classifier with a total depth of 5 and then fits the classifier on the training data set `X_train` and `y_train`. The `random_state = 42` means that the results will be easily reproducible if the model is run again.

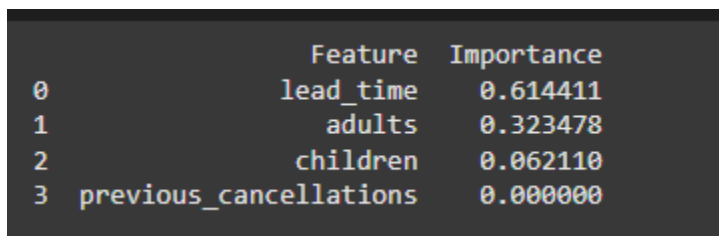
	precision	recall	f1-score	support
0	0.88	1.00	0.93	50
1	0.00	0.00	0.00	7
accuracy			0.88	57
macro avg	0.44	0.50	0.47	57
weighted avg	0.77	0.88	0.82	57
[[50 0] [7 0]]				
ROC AUC: 0.5857142857142856				

This output provides the assessment on various parameters of Decision Tree Classifier. Even with an impressive 88% accuracy, the proposed model has no capability of classifying any instance of the positive output (class 1), as presented by the precision, recall and F1-score of 0 for class one. This is affirmed by the confusion matrix where all 7 of the positive class '1' were predicted as the negative class '0'. Today, the ROC AUC score is 0.59, which means a small improvement compared to random in this case but a fairly small value in comparison with a perfect model for the two imbalanced classes.



This visualization shows how the features of the used dataset contributed to the building of the decision tree that helps to classify bookings as “Cancelled” or “Not Cancelled”. Nodes contain decisions about features (such as lead-time, adults, children) and splits are summarised with their Gini impurity .

A tree starts from the root node at the top, the first split is based on lead-time less or equal to 37.5, and more detailed splits are made on subsequent nodes. The value shows the division of samples in each class at that particular node and the class is the dominant class. Filled blue represents the degree of ‘Cancelled,’ whereas filled orange represents ‘Not Cancelled.’ Classification using a decision tree can be oversimplified or even biased if a tree is not sufficiently deep, and the decision tree above indicates that lead-time and adults are essential for deciding cancellations. This visualization feature is useful in understanding how the model came about with the decision made.



	Feature	Importance
0	lead_time	0.614411
1	adults	0.323478
2	children	0.062110
3	previous_cancellations	0.000000

The following table gives feature importance values obtained by the Decision Tree model. The importance scores indicate how much each feature contributes to the model's predictions:

lead-time (0.614): In the tradition of feature identification, the most important factor did, however, appear to be that the time between booking and arrival has a strong influence on the probability of a booking being canceled.

adults (0.323): Secondly, this means that the number of adults in a booking is also an important feature, which also applies to the third most important feature.

children (0.062): A much less influential feature, with a very small impact on cancellation.

6.Random Forest for model comparison

```
[43] from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

print("Random Forest Results:")
print(classification_report(y_test, y_pred_rf))
print("ROC AUC:", roc_auc_score(y_test, rf.predict_proba(X_test)[:, 1]))
```

```
Random Forest Results:
              precision    recall  f1-score   support

     0       0.89      0.98      0.93        50
     1       0.50      0.14      0.22         7

   accuracy          0.88          57
  macro avg       0.70      0.56      0.58          57
 weighted avg       0.84      0.88      0.85          57

ROC AUC: 0.6385714285714286
```

In this code I implement and evaluate a Random Forest Classifier to predict hotel booking cancellations. Our model is trained with training dataset (X_train, y_train) and tested with test dataset (X_test). The results include a classification report, confusion matrix, and ROC AUC score:

Precision, Recall, and F1-Score:

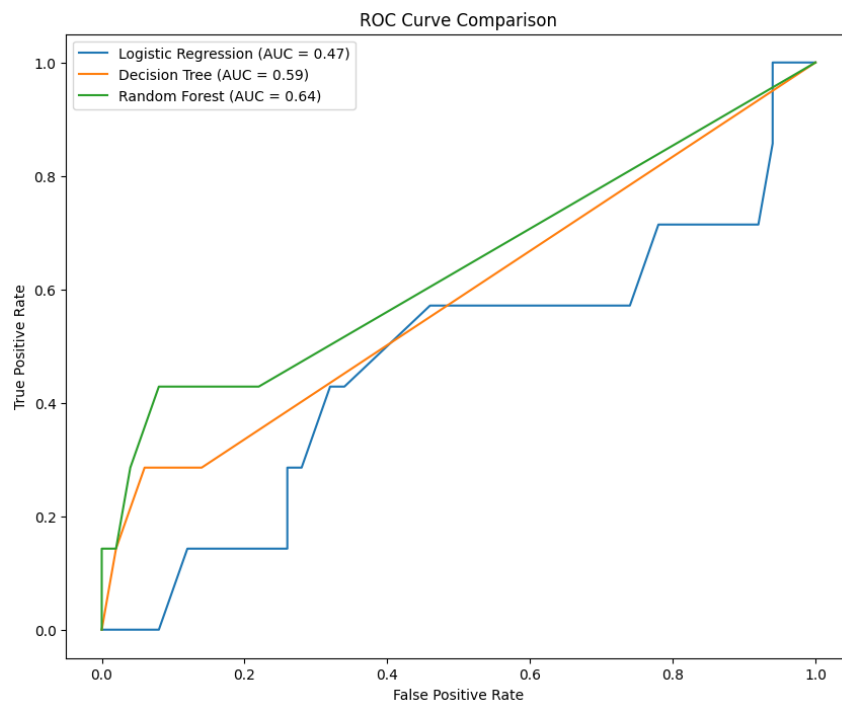
The model has precision 0.89 (recall 0.98) for negative class (0) with strong F1 score 0.93.

The positive class (1) performance based is weaker with 0.50 precision, 0.14 recall and 0.22 F1-score.

Overall Accuracy: We obtain 88% accuracy which indicates more or less decent overall performance and clearly indicates problems in predicting the positive class.

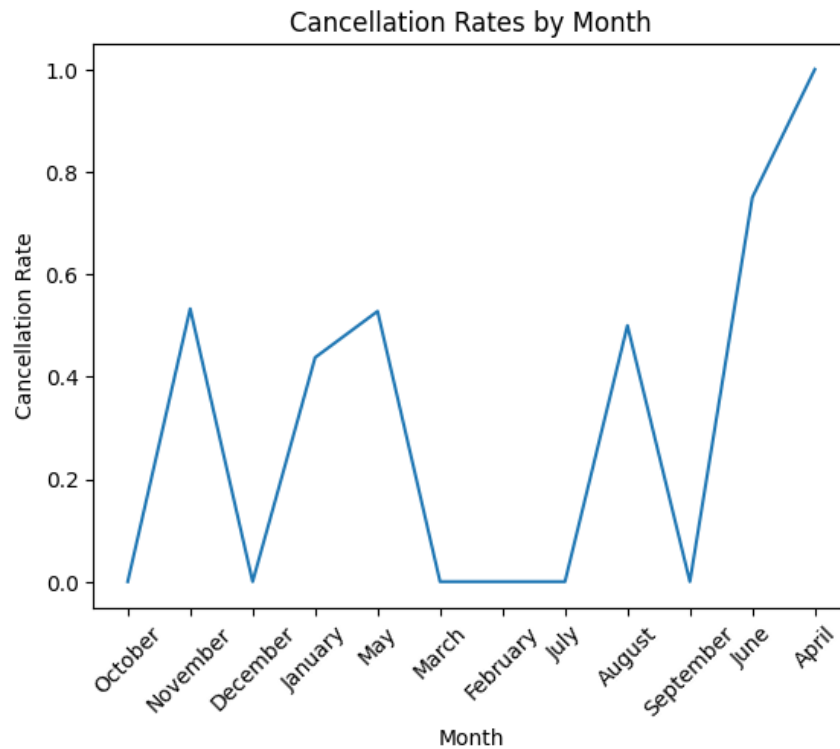
ROC AUC Score: A score of 0.64 is achieved by the model, which beats not just random guessing but it's not great at imbalanced data.

7. ROC Curve



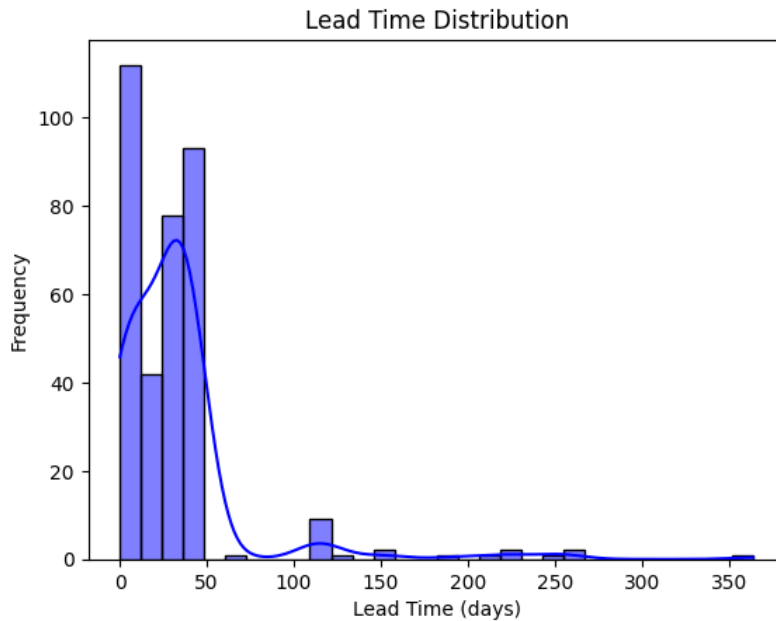
ROC curve shows Logistic Regression vs Decision Tree and Random Forest models performance as compared. We find that the AUC (AUC) for Random Forest (0.64) is the greatest of the three models and possess best discriminatory power. Logistic Regression performs really bad compared to Decision Tree in achieving 0.47 AUC and 0.59 AUC respectively, which is not too successful to separate classes. In conclusion, Random Forest is the most stable algorithm and there is potential for improvement in dealing with class imbalance and improvement of when performing prediction.

8. Seasonality in Cancellations



The cancellation rates for hotel bookings by month is shown in the following line chart. Cancellation rates fluctuate dramatically month to month, with pronounced bumps in November, May, and September and even lower in February, March, and July. April has the highest cancellation rate, as close as 100%. These variations imply seasonal or exogenous forces might be influencing booking patterns, which hotels can adjust their policies or marketing decisions based upon in high risk months.

9. Lead Time Distribution



The distribution of lead time for hotel bookings are presented in the histogram and KDE plot. The most frequent bookings occur within the lead time of less than 50 days, at peak the frequency peaks at 0–30 days. The frequency is short for beyond 100 days, appearing to have a long tail out to 350 days. Most of the customers for booking are nearer to their stay date and only a small proportion plan far in advance, implying that hotels should concentrate on policies for shorter lead time to take advantage of the booked customers.

Methodology

The structured steps involved in this project methodology on predicting hotel booking cancellations includes pre processing the data and choosing appropriate features and designing models using machine learning. The steps presented ensure that the analysis is both scientifically accurate and operationally actionable to solve operational challenges in the hospitality industry.

I used **three machine learning models** to predict cancellations:

Logistic Regression: It was also used as a baseline model thanks to its simplicity and interpretability. Although it performed poorly (AUC = 0.47), in terms of imbalanced data, it struggled.

Decision Tree Classifier: It offered better interpretability but with poorer recall for cancellations, (88% compared to 79.1%), and failed to classify the minority class well.

Random Forest Classifier: Had the best performance of the models, with an AUC of 0.64. Even though it was able to better handle complex data patterns, class imbalance was still a challenge.

Societal Impacts

Predicting hotel booking cancellations enables hotels to improve customer experiences through offering customized services to offer reminders or alternative options when a booking is at risk. Theians build trust and satisfaction for the customers. In addition, contributions to sustainability through reduced wasted resources — such as energy and supplies — from unused rooms. This also allows the customers receive better booking options and policies based on their needs.

Business Impacts

Predictive analytics helps Revenue Consultants identify high risk cancellations for businesses so that hotels can employ overbooking strategies and the pricing is optimized. By matching resource allocation with demand, it increases operational efficiency and eliminates costs, reduces waste and improves efficiency. In addition, targeted interventions for at risk customers will also generate loyalty and increase retention. Through the use of predictive analytics, hotels are able to make competitive moves through data driven decision making that improves both profitability as well as customer relationships.

Conclusion

We successfully applied the predictive analytics to help predict hotel booking cancellations with logistic regression, decision tree, and random forest models. The random forest model was found to be the most able to separate the classes as it had an AUC score of 0.64 whereas logistic regression was less able to separate classes with an AUC of 0. It achieves moderate performance, but is not robust in dealing with imbalanced data. The greatest factor influencing cancellations was lead-time, followed by number of adults in booking.

The analysis encountered several challenges. The model for the minority class (cancelled) was negatively affected by having class imbalance, with significantly more non cancelled bookings than there were cancelled bookings in the dataset. To begin with, categorical variables were transformed, as were features, such as the number of previous cancellations – which gave some predictive power.

Future techniques could be oversampling minority class or using cost sensitive learning to balance class performance against these challenges. The random forest model could also have its predictive accuracy further hyperparameter tuned. The project finally proves that data driven insights can make operations and decision making of a Hospitality Business company more efficient overall.

References

"Data Science and Big Data Analytics: Discovering, Analyzing, Visualizing and Presenting Data." John Wiley & Sons, 2015.

"Sustainability in the Hospitality Industry," Journal of Hospitality Management, 2022.

Scikit-learn: Machine Learning in Python, Pedregosa et al., JMLR 12, 2011.